Memory Organization IV – Memory Management Unit

Shobhanjana Kalita,
Dept. of CSE, Tezpur University

Slides courtesy: Computer Architecture and Organization, 9th Ed, W. Stallings

Memory Management Unit

- When the processor works on only one program at a time, it is called a **uniprogramming** system.
  - Main memory divided into two parts: 1. for OS and 2. for program currently being executed
- In a **multiprogramming** system, the processor is kept as busy as possible by allow it to work on multiple programs at a time.
  - Main memory divided into two parts: 1. for OS and 2. for all user programs
  - "user" part of memory subdivided to accommodate multiple processes.
- The task of subdivision of memory to accommodate multiple programs is carried out dynamically by the OS and is known as **memory management**

Swapping

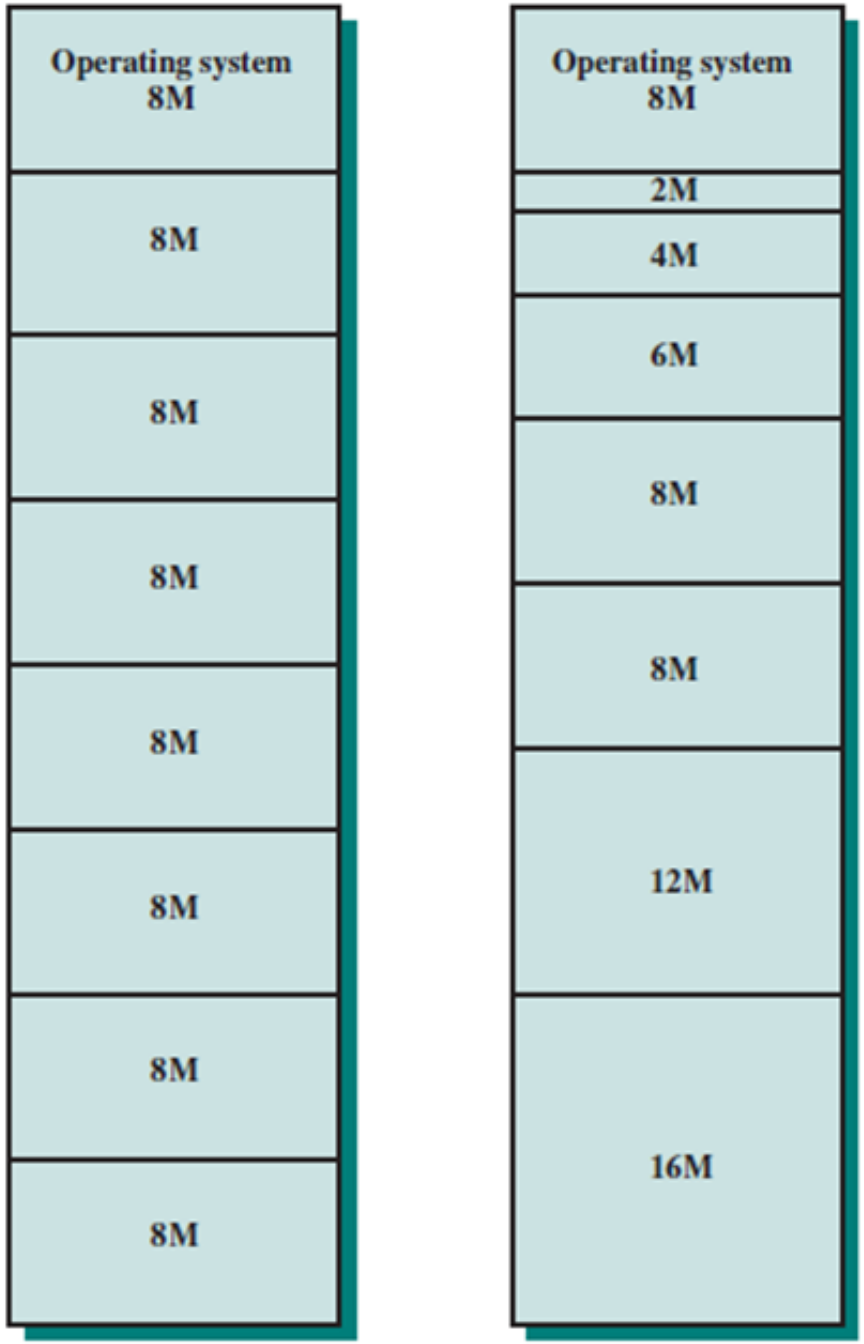
- User may have many programs to run – MMU tries to fit in these in the main memory
  - All of the programs may not be accommodated in the memory
  - Programs that are not accommodated, wait for their turn in a **queue**
- When one program ends, the memory area it holds is released and replaced by another program in the **queue**.
- When a program is waiting for an I/O operation from the user – it is occupying memory space without being executed
  - Such a program may be temporarily moved out of the memory, to accommodate some other program in the queue
  - This is called **swapping**

Logical address

- During swapping a program is moved out, and then is moved back in again to continue processing
  - May not be moved back to the same location in memory that it had occupied before
  - Thus, the *physical addresses* for instructions and data for a program keep changing – not fixed
  - *Logical addresses* are used to solve this problem
- A **physical address** is an actual location in main memory
- A **logical address** is expressed as a location relative to the beginning of the program.
  - Instructions in the program are referred to using only logical addresses.
- MMU converts from logical to physical address by adding the current starting location of the process **(base address)** to the logical address.

Partitioning

- **Fixed-size partitions** – entire memory area is divided into fixed sized partitions such that each partition can be occupied by one program
- Two types-
  - Equal size partitions
  - Unequal size partitions – when a program comes in MMU allocates the smallest available partition that can hold it
- **Disadv** – memory wastage



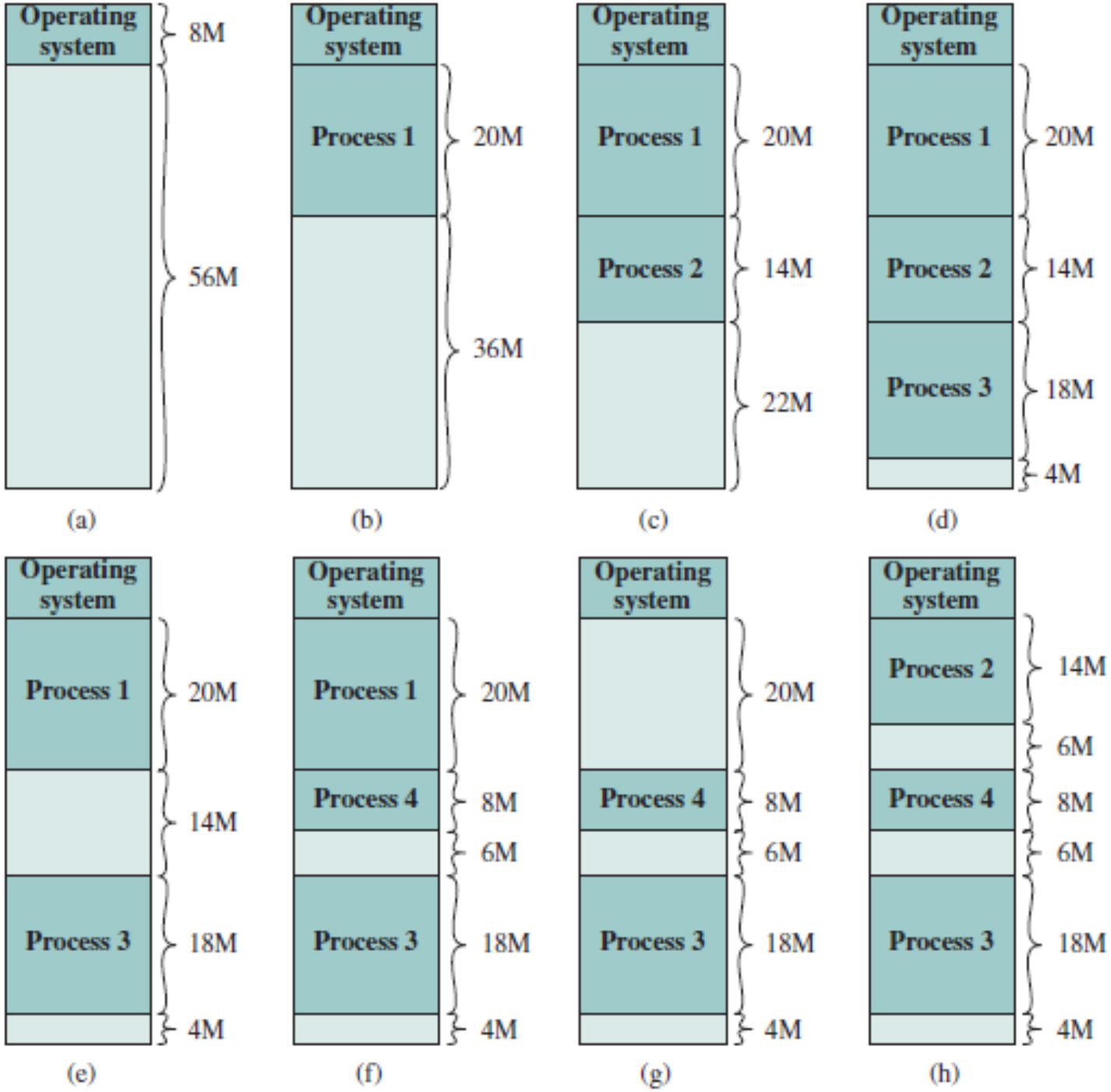
(a) Equal-size partitions (b) Unequal-size partitions

Partitioning

- **Variable sized partitioning** - When a program is brought into memory, it is allocated exactly as much memory as it requires and no more
- **Disadv** – Over time it leads to a situation in which there are a lot of small holes in memory that cannot be efficiently utilized – this is **fragmentation**
- One solution is **compaction** - From time to time, the OS shifts processes in memory to place all the free memory together in one block.
  - time-consuming procedure, wasteful of processor time.

Partitioning

Effect of variable sized partitioning

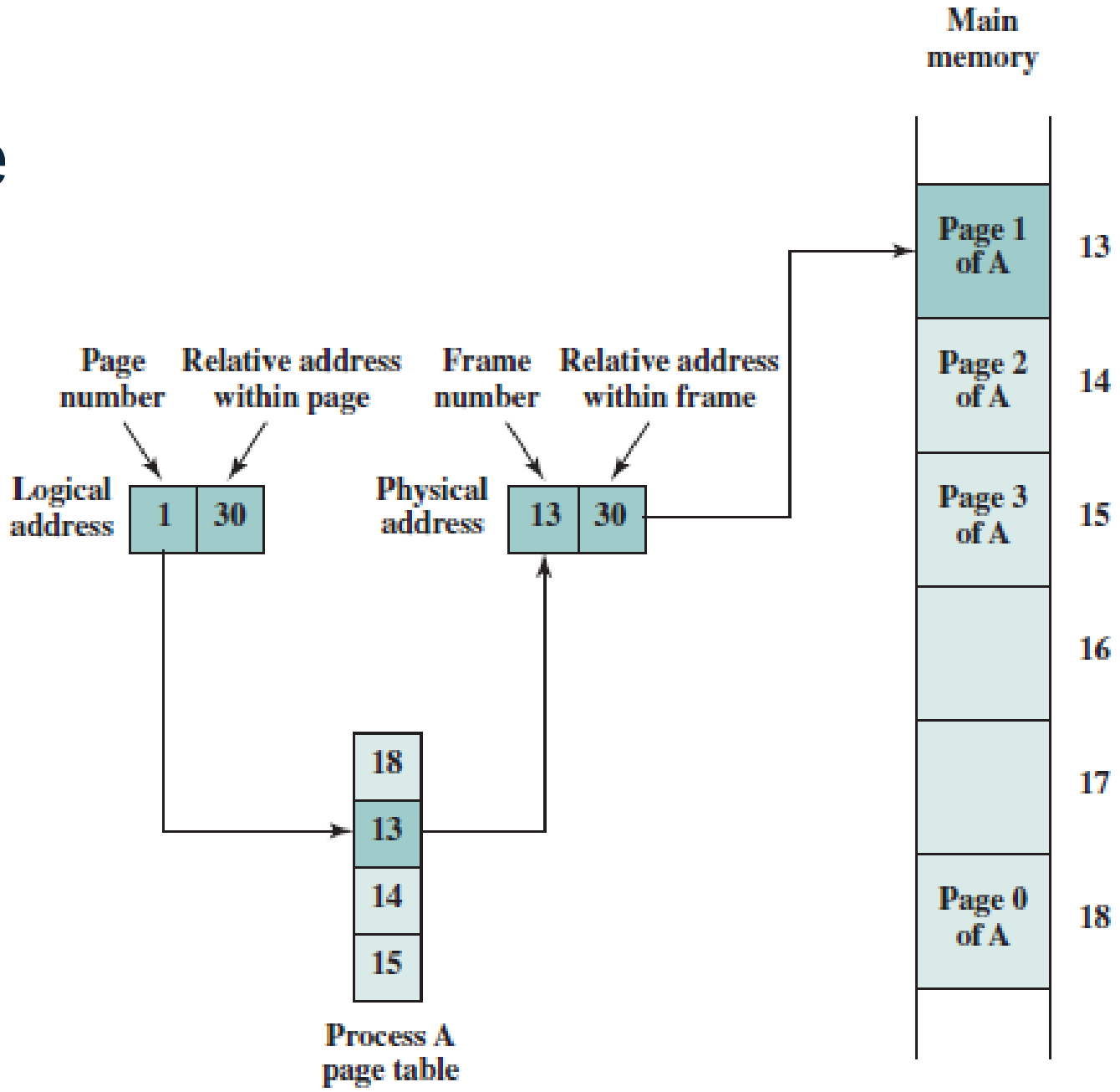


Paging

- Both unequal fixed-size and variable-size partitions are inefficient in the use of memory
- Paging is the approach where –
  - memory is partitioned into relatively small, equal fixed-size chunks – called **frames**
  - each program is also divided into small fixed-size chunks of some size – called **pages**
  - A page of program occupies a single frame in memory
  - The entire set of pages of a program may or may not occupy continuous set frames in memory
  - OS maintains a **page table** for each program, that shows the frame location for each page of the process

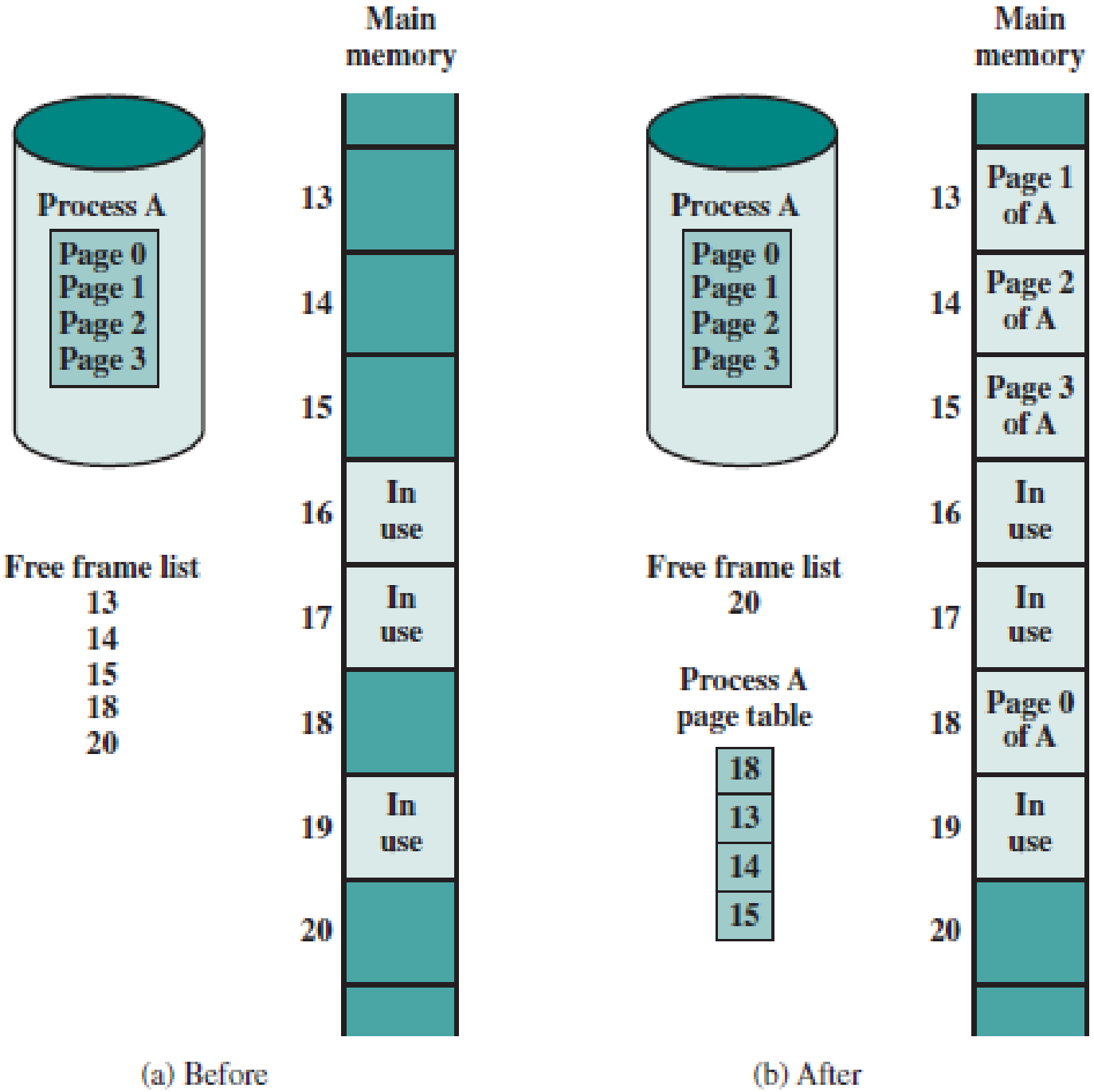
Paging – Page table

- In the **page table** for each process, frame location for each page of the process is given
- **Logical address** consists of a page number and a relative address within the page (offset)
- **Physical address** consists of a frame number and relative address (offset)



Paging

Pages for a program is allocated from a **free frames list** maintained by OS

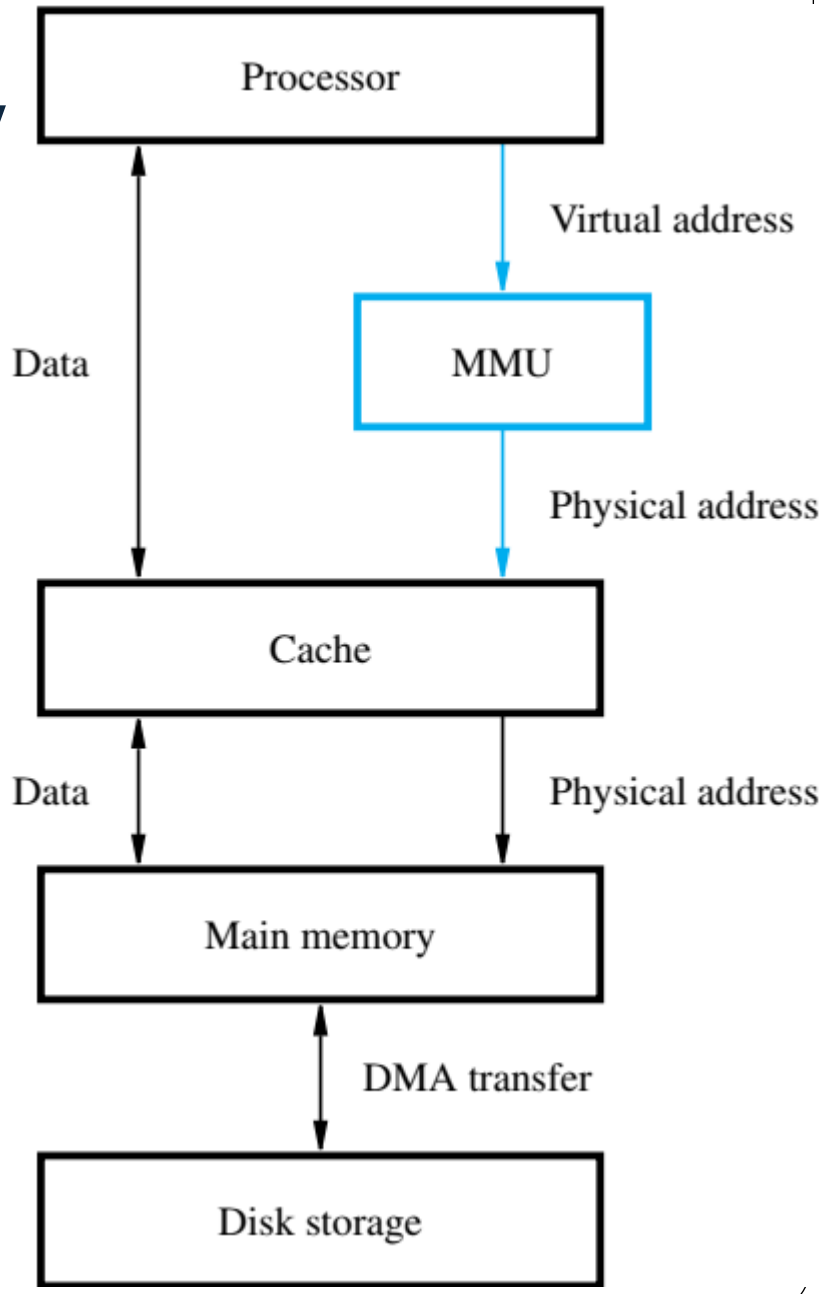


Demand Paging

- In simple paging – when a program is brought into memory, all its pages are fitted into free frames in the memory
- In **demand paging** – each page of a program is brought in only when it is needed, that is, on demand
  - Entire program may not physically be in memory during execution – only a few pages are loaded as demanded by CPU
  - Principle of **locality of reference** is used in choosing which pages are to be loaded
  - If the program references data on a page not in memory, a **page fault** is triggered – OS then loads appropriate page from disk onto memory
  - When it brings one page in, it must throw another page out –this is **page replacement**

Demand Paging–Virtual Memory

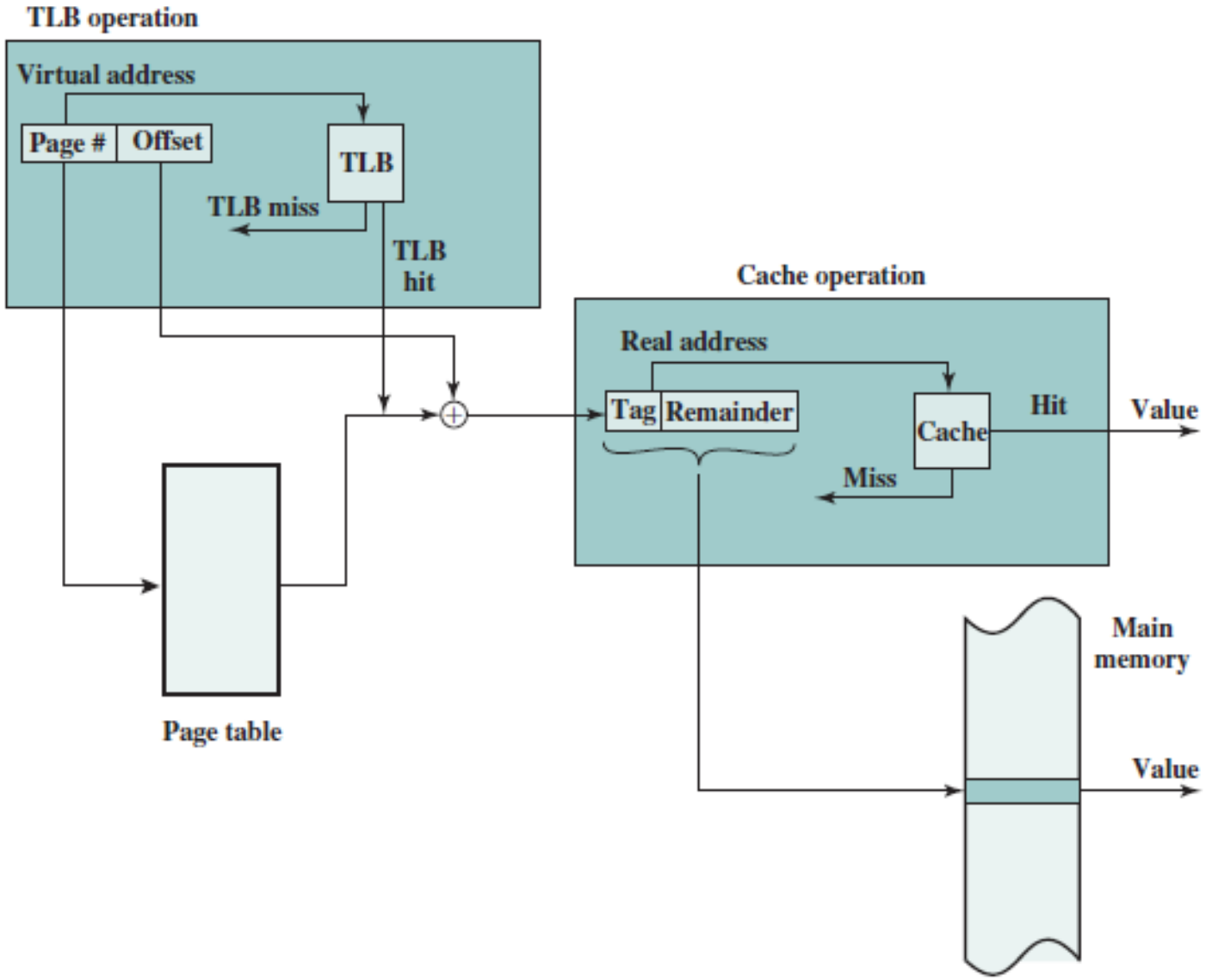
- With demand paging, it is not necessary to load an entire process into main memory, allowing
  - More number of programs to be in the memory
  - A program can be larger than all of main memory
- The details of loading the appropriate pages of a program under execution into the **real memory** is left to the OS and the hardware
  - The programmer/user simply perceives a very large memory – part of which is allocated from the disk
  - The part of disk memory that is used by programmer/user in lieu of main memory is referred to as **virtual memory.**
- Virtual memory allows for very effective multiprogramming
  - Relieves the user of the tight constraints of main memory



Demand Paging – TLB

- Ideally page table should be accommodated in the MMU, however the page tables may be too large; page table is instead kept in the main memory
- To read a word from memory –
  - **Translate** logical address to physical address **using page table**
  - Page number + offset -> frame address + offset
- Every memory reference requires **two** physical memory accesses
  - To fetch the appropriate page table entry
  - To fetch the desired data
- To overcome this – use a special cache for page table entries is included in the MMU – called **translation lookaside buffer (TLB)**
  - Contains a copy of a small portion of the page table
  - Consists of page table entries that corresponds to most recently used pages

Demand Paging – TLB with Cache



Demand Paging

- CPU requests for some logical address (page number + offset)
  - MMU consults the TLB to see if requested page table entry is present
    - If yes, the physical address obtained (frame number + offset)
    - If not, the entry is accessed from a page table in memory
  - If physical address is generated (consisting of **tag + remainder**), cache is consulted to see if the block containing the word is present
    - If so, it is returned to the processor
    - If not, the word is retrieved from main memory
  - If requested address corresponds to a page not in main memory, then retrieve the entire page from disk – this called a *page fault*.
    - MMU raises an exception (interrupt), and control is transferred to OS to fetch the page from disk
    - If there are no free frames in the memory, replacement policies (LRU, FIFO etc) are used to determine which page will be replaced